Intelligent Fog Orchestration System for Distributed IoT Workload Management

Irfan Uruchi
Faculty of Contemporary Sciences and Technologies
South East European University
Tetovo, North Macedonia

***Abstract*—This paper presents a fog orchestration system for running distributed IoT and edge workloads across multiple Docker-capable nodes. The system uses a Fog Controller, MQTT broker, IoT Smart Nodes, Docker workload containers, telemetry messages, token-protected commands, and desired-state reconciliation.**

The implemented setup uses two fog nodes. The first node runs ProMatch and FluidLab CFD. The second node runs Integral Calculator and CNN Edge Classifier. The controller publishes the desired workload state over MQTT. Each IoT Smart Node receives the state assigned to its own node ID, checks the local Docker runtime, and starts missing managed workloads when the actual state does not match the expected state.

The project also includes a Fog Intelligence LLM service. This service is used by the Fog Controller for operational summaries through a local Ollama runtime. It is separate from the Integral Calculator explanation path, which uses a Cerebras API key inside the workload environment. The result is a working fog orchestration prototype that combines MQTT communication, Docker workload control, node telemetry, reconciliation, CNN edge inference, and controller-level LLM support.

***Index Terms*—fog computing, IoT, MQTT, Docker, edge computing, desired-state reconciliation, CNN, MobileNetV2, workload orchestration**

I. Introduction

IoT systems often need computation near the devices or networks where data is produced. Sending every task to a remote cloud is not always the best option, especially when latency, bandwidth, local availability, or network reliability matters. Fog computing provides an intermediate layer where workloads can run closer to the edge while still being controlled from a central system.

This project implements an Intelligent Fog Orchestration System for distributed IoT workload management. The system is not only a dashboard. It includes a controller-node architecture, MQTT-based communication, Docker workload deployment, telemetry reporting, token-protected control, runtime monitoring, desired-state reconciliation, and a local LLM-based operational summary service.

The main runtime idea is simple: the controller stores what should be running, and each node checks what is actually running. If the two states do not match, the node corrects its local Docker runtime. This gives the system a control loop instead of depending only on one-time start commands.

The project uses four different workloads to show that the orchestration layer is not tied to one application type. ProMatch represents Prolog-based reasoning. FluidLab CFD represents a scientific simulation workload. Integral Calculator represents symbolic computation and explanation generation. CNN Edge Classifier represents MobileNetV2-based image inference at the fog layer.

The main contributions are:

- a working controller-node fog orchestration platform;
- MQTT-based communication between the controller and IoT Smart Nodes;
- Docker-based deployment of heterogeneous workloads;
- desired-state reconciliation on the node side;
- telemetry and event reporting back to the controller;
- LAN and Tailscale-compatible remote node deployment;
- controller-level operational summaries through a local Ollama-based LLM service;
- CNN edge inference as one of the managed workloads.

II. Background and Related Work

Fog computing moves part of the computation closer to the edge of the network. This is useful in IoT environments where data may be generated continuously and where sending all processing to the cloud can increase latency or bandwidth usage [1], [2].

MQTT is a lightweight publish-subscribe protocol used in many IoT systems. Instead of connecting every device directly to every other device, MQTT clients communicate through a broker [3]. In this project, the Fog Controller and IoT Smart Nodes communicate through MQTT.

Docker is used because the workloads are heterogeneous. The workloads in this project are not written with the same language or framework, but they can all be packaged and managed as containers. The node agent only needs to know the image, ports, environment variables, and managed labels for each workload [4].

Desired-state reconciliation is a common idea in orchestration platforms. The system stores the expected state, observes the actual state, and applies corrections when the two differ. Large orchestration systems such as Kubernetes use this pattern at a larger scale [5]. This project applies the same idea in a smaller fog computing setup with custom IoT Smart Nodes.

III. System Architecture

The system is built around a Fog Controller, MQTT broker, IoT Smart Nodes, Docker workload containers, and a Fog Intelligence LLM service.

There are two different paths in the architecture. The first path is the main orchestration path:

```
Fog Controller
-> MQTT Broker
-> IoT Smart Nodes
-> Docker workloads
```

The second path is the operational summary path:

```
Fog Controller
-> Fog Intelligence LLM
-> local Ollama
```

The Fog Intelligence LLM service does not deploy containers. It is not in the MQTT node control path. The controller uses it only for generating operational summaries from controller-level information.

A. Fog Controller

The Fog Controller is the central interface of the system. It stores the desired workload state for each node, publishes updates through MQTT, receives telemetry, shows node status, records orchestration events, and requests operational summaries.

The controller does not directly start containers on remote machines. Container execution is done by the IoT Smart Node running on each fog node.

B. MQTT Broker

The MQTT broker connects the controller with the IoT Smart Nodes. It carries desired-state messages, telemetry, status updates, and orchestration events.

This design keeps the nodes independent from direct HTTP calls. The same setup can run on one machine, across a LAN, or across remote machines connected through Tailscale by changing the MQTT host.

C. IoT Smart Node

The IoT Smart Node is the node-side agent. Each node starts with a fixed identity. In the implemented setup, the node IDs are:

```
node-1
node-2
```

Each node connects to MQTT, listens for desired-state updates, checks whether the update applies to its own ID, inspects the local Docker runtime, and starts or stops managed workload containers.

The node requires access to the host Docker socket:

```
/var/run/docker.sock
```

This is required because the node controls workload containers on the host Docker engine.

D. Fog Intelligence LLM

Fog Intelligence LLM is a helper service used by the Fog Controller. It connects to local Ollama through:

```
OLLAMA_BASE_URL
OLLAMA_MODEL
```

The service generates operational summaries for the controller. It does not use Cerebras and does not communicate directly with the IoT Smart Nodes.

E. Integral Calculator Explanation Path

The Integral Calculator has a separate explanation path. It receives a Cerebras API key through the environment of the node where the workload runs.

```
IoT Smart Node 2
-> Integral Calculator
-> Cerebras explanation support
```

This is separate from the Fog Intelligence LLM service.

IV. Design Decisions

MQTT was used instead of direct controller-to-node HTTP calls. This keeps communication simple and makes the deployment more flexible. Nodes only need to reach the broker, so they can run locally, on another machine in the LAN, or through Tailscale.

Docker was used as the workload execution layer because the project workloads are different from each other. ProMatch, FluidLab CFD, Integral Calculator, and CNN Edge Classifier do not share one implementation stack. Running them as containers gives the IoT Smart Node one common interface for starting, stopping, and checking them.

The Fog Intelligence LLM service was kept outside the deployment path. The controller uses it for summaries, but workload control remains deterministic. This avoids making the LLM responsible for starting or stopping services.

The reconciliation logic was placed on the node side. This means the node can correct its own local Docker runtime after receiving the desired state. The controller defines what should run, while the node is responsible for applying that state locally.

V. Implementation

The implementation is split across several repositories. The main repository is the project entry point and links the component repositories, Docker images, documentation, paper, presentation, and run checklist.

A. Repositories

- Main project: https://github.com/IrfanUruchi/intelligent-fog-orchestration-system
- Fog Controller: https://github.com/IrfanUruchi/fog-controller
- IoT Smart Node: https://github.com/IrfanUruchi/iot-smart-node

- Fog Intelligence LLM: https://github.com/IrfanUruchi/fog-intelligence-llm
- CNN Edge Classifier: https://github.com/IrfanUruchi/cnn-edge-classifier

B. Docker Images

The project uses these Docker images:

- `irfanuruchi/fog-controller:latest`
- `irfanuruchi/iot-smart-node:latest`
- `irfanuruchi/fog-intelligence-llm:latest`
- `irfanuruchi/promatch:latest`
- `irfanuruchi/fluid-lab-cfd:cpu`
- `irfanuruchi/integral-calculator:latest`
- `irfanuruchi/cnn-edge-classifier:latest`

The deployable images were published for both `linux/amd64` and `linux/arm64`. This allows the system to run on standard x86 machines and ARM-based devices.

C. Runtime Configuration

The system is configured through environment variables. For local Docker Desktop usage, the MQTT broker host can be configured as:

```
MQTT_HOST=host.docker.internal
MQTT_PORT=1883
```

For LAN deployment, the MQTT host is replaced with the broker machine LAN IP. For Tailscale deployment, it is replaced with the broker machine Tailscale IP.

Node identity is configured through `NODE_ID`. The Fog Intelligence LLM service uses `OLLAMA_BASE_URL` and `OLLAMA_MODEL`. The Integral Calculator receives `CEREBRAS_API_KEY` through the node environment because that workload uses it separately for explanation generation.

VI. Desired-State Reconciliation

The main intelligent behavior of the system is desired-state reconciliation.

The project uses this desired-state placement:

```
node-1:
  - promatch
  - fluidlab

node-2:
  - integral-calculator
  - cnn-edge-classifier
```

Each IoT Smart Node receives the desired state through MQTT and reads only the part assigned to its own node ID. The node then compares the expected workload list with the actual local Docker runtime.

If the local runtime already matches the desired state, the node does not need to change anything. If a required managed workload is missing or stopped, the node starts it. If a managed workload should not run on that node anymore, the node can stop it according to the configured behavior.

For example, if `node-1` should run ProMatch and FluidLab CFD, but FluidLab CFD is stopped, the node detects the mismatch and starts FluidLab CFD again.

This is the main difference between the system and a simple start script. The node keeps checking the runtime state instead of assuming that the first command succeeded forever.

VII. Workload Case Studies

A. ProMatch

ProMatch is a Prolog-based reasoning and matchmaking workload. It demonstrates that the orchestration system can manage a logic-based service as a fog workload. In the project configuration, ProMatch runs on `node-1`.

B. FluidLab CFD

FluidLab CFD is a scientific simulation workload. The project uses the CPU image `irfanuruchi/fluid-lab-cfd:cpu`. This workload represents a heavier service running at the fog layer instead of being treated as a simple web page.

C. Integral Calculator

The Integral Calculator is a Haskell-based symbolic computation workload. It supports parsing, simplification, differentiation, symbolic integration, numerical integration, plotting, and explanation generation. In the project configuration, it runs on `node-2`.

The explanation feature uses `CEREBRAS_API_KEY`. This is separate from the Fog Intelligence LLM service.

D. CNN Edge Classifier

The CNN Edge Classifier is a MobileNetV2-based image classification workload using ImageNet pretrained weights [6]. It provides a web interface and a prediction API. It accepts an uploaded image, runs inference, and returns predictions, confidence scores, and inference time.

This workload was added to demonstrate CNN/DCNN inference as a managed edge workload.

VIII. Demonstrated Runtime Scenario

During the project run, the Fog Controller was used to assign two workloads to each IoT Smart Node. The first node, `node-1`, was assigned ProMatch and FluidLab CFD. The second node, `node-2`, was assigned Integral Calculator and CNN Edge Classifier.

After the desired state was applied, each node inspected its local Docker runtime and started the required workload containers. The controller displayed node connectivity, workload status, telemetry updates, and orchestration events.

The CNN Edge Classifier was accessed through its web interface and used to run MobileNetV2 inference on an uploaded image. The Integral Calculator was used as the symbolic computation workload. The Fog Intelligence LLM summary was generated from the controller through the local Ollama-based service.

IX. Evaluation

The system was evaluated through a working multi-node setup. The evaluation focused on whether the system could connect nodes, apply desired workload placement, start heterogeneous workloads, report telemetry, and recover from a managed workload mismatch.

A. Node Connectivity

The Fog Controller and IoT Smart Nodes communicate through MQTT. After startup, the controller shows `node-1` and `node-2` as connected and receives telemetry from both nodes.

B. Workload Placement

The expected workload placement is shown in Table I.

TABLE I
WORKLOAD PLACEMENT USED IN THE PROJECT

Node	Assigned workloads
node-1	ProMatch, FluidLab CFD
node-2	Integral Calculator, CNN Edge Classifier

This confirms that workload placement is controlled through the controller-defined desired state instead of by manually starting each service.

C. Reconciliation Test

A reconciliation test can be performed by manually stopping a managed workload. The IoT Smart Node detects that the actual Docker runtime no longer matches the desired state and starts the missing workload again.

D. Heterogeneous Workloads

The platform manages different workload categories: logic reasoning, scientific simulation, symbolic computation, explanation generation, and CNN inference. The orchestration layer does not depend on the internal implementation of each workload.

E. Multi-Architecture Images

The main deployable images were published for `linux/amd64` and `linux/arm64`. This is useful for fog environments where different nodes may run on different hardware architectures.

F. Operational Summary

The Fog Controller can request an operational summary from Fog Intelligence LLM. The service uses local Ollama and returns a summary of controller-level system state. This supports the operator but does not replace the deterministic orchestration logic.

X. Security and Limitations

The system uses token-protected command handling through `DEVICE_TOKEN`. Runtime secrets and keys are passed through environment variables. The implementation also includes rate limiting, burst limiting, restricted configuration updates, Docker-managed labels, and controlled workload handling.

The IoT Smart Node requires Docker socket access because it manages local workload containers. This gives the node agent strong control over the host Docker runtime, so the node agent must be treated as a trusted component.

The current implementation is still a project prototype. It uses a small number of nodes, a fixed workload placement, and a limited scheduling policy. It does not implement automatic workload migration or long-term distributed telemetry storage. These limitations are acceptable for the current project because the goal is to demonstrate the complete fog orchestration flow and not to replace a production orchestrator.

XI. Conclusion

This paper presented the Intelligent Fog Orchestration System, a distributed platform for managing IoT and edge workloads across fog nodes. The system combines a Fog Controller, MQTT broker, IoT Smart Nodes, Docker workload deployment, telemetry reporting, desired-state reconciliation, remote connectivity, and LLM-assisted operational summaries.

The implemented system shows that a practical fog orchestration layer can be built using a controller-node model. The controller defines the desired workload state, while each node applies that state locally through Docker. The desired-state reconciliation loop provides the main intelligent runtime behavior by detecting mismatches and restoring required workloads.

The project was demonstrated using four heterogeneous workloads: ProMatch, FluidLab CFD, Integral Calculator, and CNN Edge Classifier. These workloads show that the same orchestration model can manage reasoning systems, scientific simulation, symbolic computation, and CNN inference.

Future work can extend the platform with resource-aware placement, persistent event storage, automatic workload migration, richer telemetry, and larger multi-node deployments.

References

[1] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012.

[2] OpenFog Consortium, "OpenFog reference architecture for fog computing," 2017.

[3] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, "MQTT Version 5.0," OASIS Standard, 2019.

[4] Docker Inc., "Docker documentation." [Online]. Available: https://docs.docker.com/

[5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[7] Ollama, "Ollama documentation." [Online]. Available: https://ollama.com/

[8] Eclipse Foundation, “Eclipse Mosquitto.” [Online]. Available: https://mosquitto.org/